UPDATED PROJECT PLAN GROUP 2

Table of Contents

1. Introduction 3
   Project Organization 3
2. Project Scope 5
   Final Scope Update 5
3. Risk Analysis 6
4. Hardware and Software Requirements 8
   Hardware Requirements 8
   Software Requirements 8
5. System Diagrams (Use Case & Application Navigation) 9
   Use Case Diagram: 9
   Application Navigation Diagram: 9
6. Work Breakdown Structure 11
   Requirement-Based WBS Table 12
7. Project Schedule 13
8. Gantt Chart 14
   Final Schedule Update 15
9. Monitoring and Reporting Mechanisms 15
   Final Monitoring Update 15
10. Final Project Plan Update 15
11. Appendix 16

1.Introduction

The purpose of this project was to develop a functional prototype of a calorie-tracking web application that helps users monitor their daily food intake and manage personal calorie goals. The application allows users to create a profile, set a daily calorie target, record foods and meals, view calories consumed and remaining, and review previous calorie history. These features provide users with a simple and organized way to monitor their eating habits.

The project focused on creating a clear and easy-to-use interface, accurate calorie calculations, and basic progress tracking. The system was developed as a responsive web application that can be used through a standard web browser while remaining usable on smaller screen sizes.

The development team followed the Software Development Life Cycle to plan, design, build, test, document, and prepare the working Lifestyle Guru prototype for final submission. The project was completed within the course timeline using free or commonly available development tools. The final phase focused on system integration, regression testing, bug correction, documentation updates, and preparation of the final submission package.

Project Organization

Member	Role	Description
Tri Phong	Project Manager/ Team lead	Organize team meetings, assign and track tasks, monitor deadlines, make sure everyone is contributing, coordinate communication between members, and keep the project on schedule.

Craig Curry	Requirements/ System Analysis	Define and document the app's requirements, features, business rules, and use cases. Maintain requirement updates and help make sure the final system matches what the team originally planned.
Aliya Kerimkesheva	UI/UX Developer	Design and build the screens, navigation, forms, buttons, and visual layout. Make sure the app is easy to understand, consistent, and simple for users to navigate.
Kyler Reed	Back-End/Data Development	Handle how user accounts, food entries, calorie goals, meal logs, and history are stored, retrieved, updated, and validated. Develop the data-management side of the application and test those functions.
Robert Casey	Software Development Lead	Design the overall program structure, develop the main application logic and calorie calculations, guide coding decisions, help integrate different parts of the program, and make sure the code is organized and reusable.
Tri Phong	QA/testing Manager	Create and organize test cases, test the full application for bugs and incorrect behavior, verify calorie calculations and input validation, record test results, and collect screenshots/evidence for the testing documentation.

2.Project Scope

The project will focus on developing a functional prototype of a responsive calorie-tracking web application. The purpose of the prototype is to demonstrate the main features and user workflow of a calorie-tracking system without attempting to create a full commercial product.

The core features included in the project will be:

- User profile management
- Daily calorie goal setup
- A small food and nutrition database
- Ability to add custom food entries
- Meal logging for breakfast, lunch, dinner, and snacks
- Daily calorie and basic nutrition totals
- A dashboard showing calories consumed, remaining, or over the daily goal
- History of previous daily calorie records
- Basic weekly progress or summary information
- Ability to add, edit, and delete food or meal entries
- Basic input validation and error handling

The prototype will be developed as a web application using Python, Flask, SQLite, HTML, CSS, Bootstrap, and Figma for UI/UX design. The interface will be designed to work well in a standard web browser and remain usable on smaller screen sizes.

To keep the project realistic for the team's current skill level and limited development timeline, the prototype will use a manageable sample food database rather than a large commercial nutrition database. Advanced features such as AI food recognition, barcode scanning, wearable-device integration, social networking, cloud synchronization, and complex medical or dietary recommendations will not be included in the initial version.

Final Scope Update

The final Lifestyle Guru prototype preserves the main calorie-tracking workflow described in the original project scope. During implementation, several features were revised to keep the project within the available course timeline and technical scope. The

originally planned basic weekly summary was not implemented as a separate feature; the final prototype instead provides date-based nutrition history and daily totals. The final version uses one active profile rather than multiple profiles, simplified login validation, a reset option instead of permanent goal deletion, and no JSON-based recommendation system. The completed prototype includes profile onboarding and editing, BMI/BMR calculations, calorie and macronutrient goals, Dashboard progress tracking, Food Library management, automatic calorie calculation, Favorites, Most Common Foods, meal logging, meal history, and date-based nutrition tracking.

3.Risk Analysis

Risk	Probability	Effect	Mitigation
Late Delivery	Low: The deadline is known in advance, although unexpected development or testing problems could cause delays.	Serious: An incomplete application could result in missing required features and negatively affect the project grade.	The team will create a reasonable project schedule, prioritize core calorie-tracking features, monitor progress regularly, and postpone optional features if they threaten completion.
Cloud Software / Data Loss	Low: Established cloud services are generally reliable, but outages or file-management mistakes can still occur.	Serious: Losing source code, documentation, or project files could delay development and require work to be recreated.	The team will keep the technology stack simple, document the development environment, use version control, and test calculations, database operations, and major features throughout development.
Team Member Not Available	Low: There is a relatively low chance that a team member will be unavailable for a long period.	Serious: Missing contributions could delay dependent tasks and increase the workload for other team members.	The team will use shared documentation and GitHub version control so project work is not dependent on one team member. Team members will maintain enough shared knowledge to continue work when needed.

Scope Creep	High: A calorie-tracking application could easily expand as the team identifies additional features.	Serious: Extra features could consume development and testing time and prevent completion of the core application.	The team will clearly define Version 1 of the Calorie Tracker. Features outside the reasonable starting scope will not be added unless their effect on time and resources is evaluated.
Calorie Calculation Errors	Moderate: Calculation and data-entry errors are possible during development.	Serious: Incorrect results could reduce user trust and cause required tests to fail.	Calorie formulas and calculations will be tested with known values and multiple test cases. Incorrect results will be corrected before final integration.
Food Database Limitations	Moderate: The starting scope uses a manageable database rather than a large commercial nutrition database.	Tolerable to Serious: Users may not find the food they need, and inaccurate data could affect meal totals.	The team will begin with a manageable food database and allow custom food entries so users can add foods that are not already available.
Software / Development Environment Failure	Moderate: Development environments can experience configuration, dependency, or software errors.	Serious: A failure could temporarily prevent development and require time to restore the environment.	Development tools and dependencies will be documented so the environment can be recreated if necessary. Project files will also be backed up regularly.
Performance / Database Issues	Low: The starting scope is small, but database or implementation problems are still possible.	Tolerable to Serious: Poor performance could make the application difficult to use and cause testing problems.	The team will test database queries and application performance during development and simplify or correct inefficient operations if problems are found.
User Input / Validation Errors	Moderate: The application depends on several	Serious: Invalid input could produce incorrect calorie goals or meal totals.	Input validation will be added to forms to prevent missing, invalid, or unreasonable values. Invalid input will

	numerical inputs and forms.		display an error message instead of being processed.
Presentation Cannot Be Completed	Moderate: Completing the application may receive higher priority than preparing the presentation.	Serious: An incomplete presentation could negatively affect the final project grade.	The presentation will be started before the final week, divided among team members, and reviewed before the deadline so it does not depend on last-minute preparation.

4.Hardware and Software Requirements

Hardware Requirements

a. Desktop or laptop computer

b. Monitor or built-in laptop display

c. Keyboard

d. Mouse or trackpad

e. Internet connection for team collaboration, GitHub, and downloading development tools

f. Sufficient storage space for the project files, development software, and local database

Software Requirements

g. Python – used for the main application logic and calorie calculations

h. Flask – used as the web framework connecting Python to the web interface

i. SQLite – used to store user profiles, food information, meal logs, calorie goals, and history

j. HTML – used to structure the application pages

k. CSS – used to style the application

l. Bootstrap – used to help create a clean and consistent interface

m. JavaScript – used only when needed for basic interactive features

n. Figma – used to design UI/UX layouts, wireframes, and screen mockups before implementation

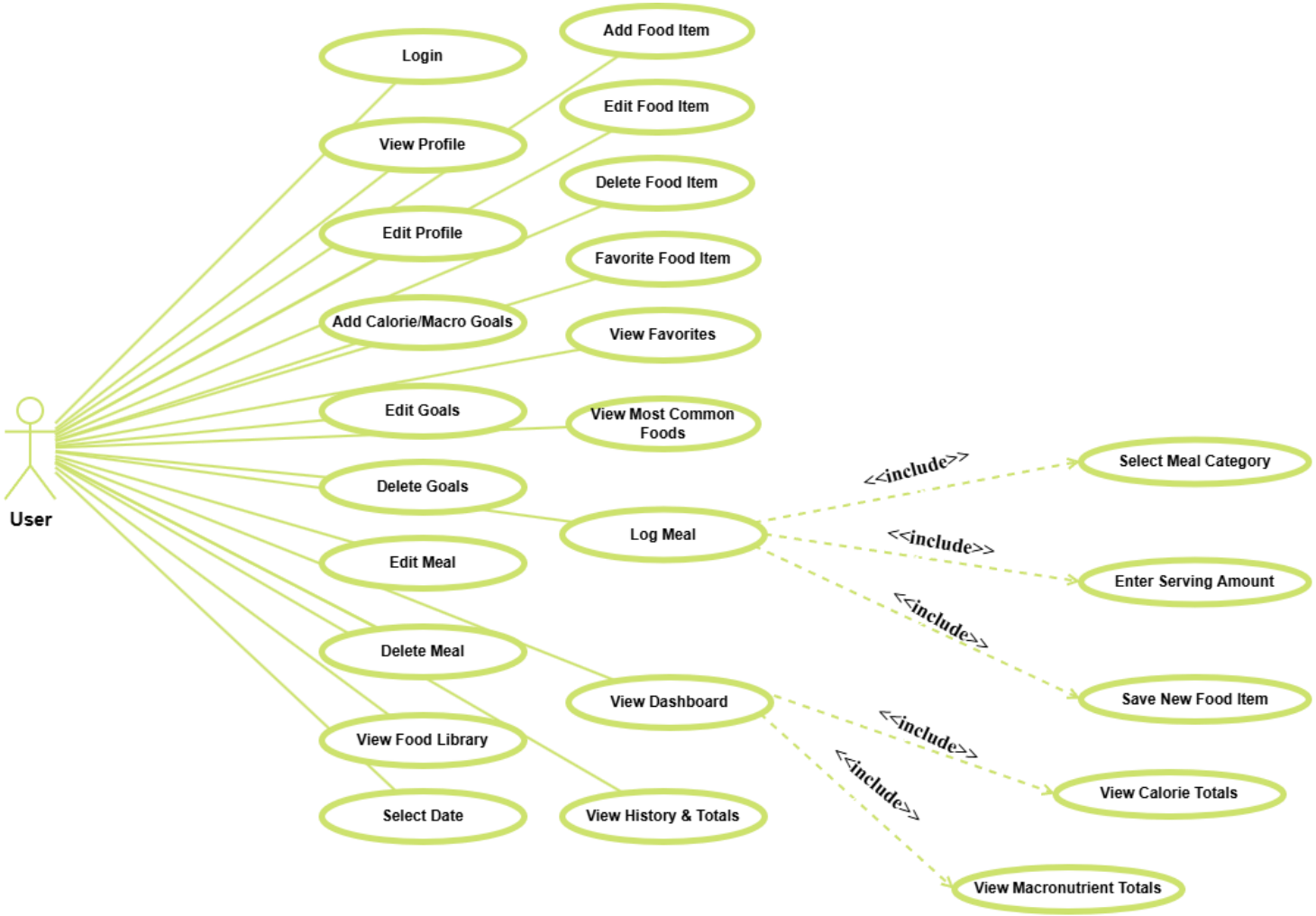
o. Visual Studio Code – used to write and manage project code

p. Git and GitHub – used for version control and team collaboration

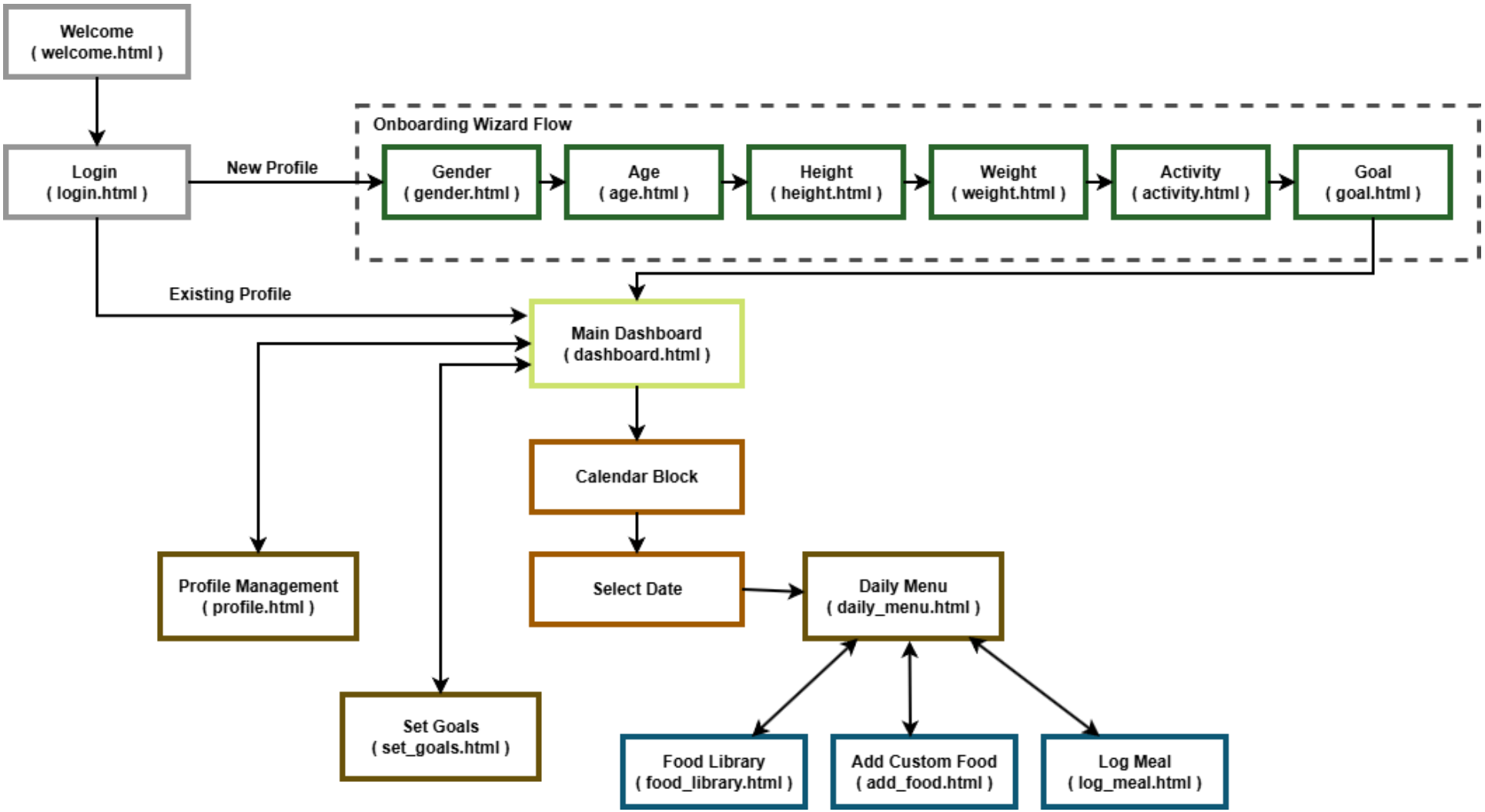
q. Web browser such as Chrome, Edge, or Firefox – used to run and test the application.

5.System Diagrams (Use Case & Application Navigation)

Use Case Diagram:

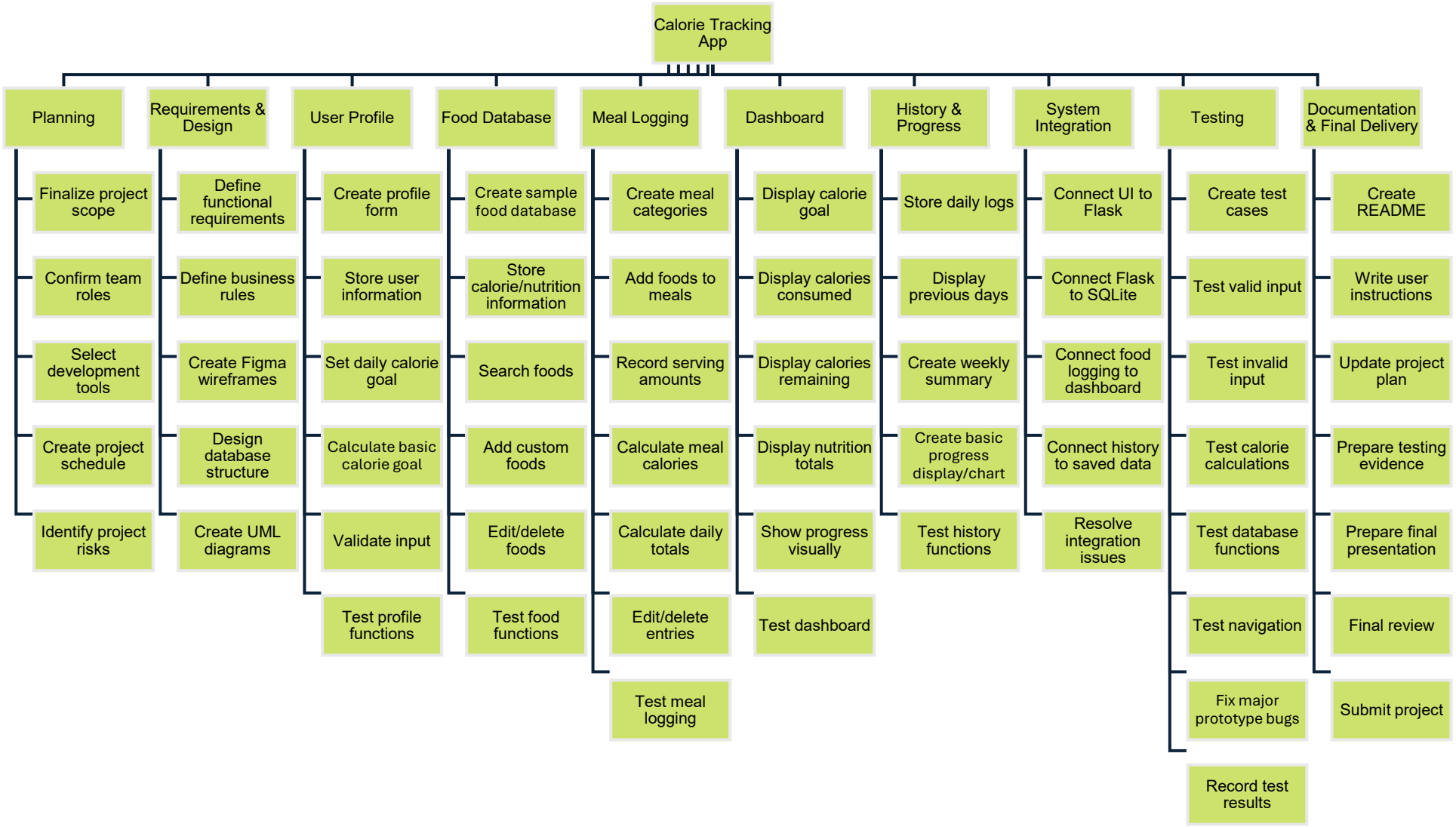


Application Navigation Diagram:



6.Work Breakdown Structure

The original WBS below shows the project's high-level work categories. To provide clearer requirement traceability, the updated requirement-based WBS table that follows maps each major system requirement to its implementation activity, validation activity, responsible team member(s), and final status.



Requirement-Based WBS Table

Requirement / Feature	Implementation Activity	Validation Activity	Responsible Member(s)	Final Status
Welcome Screen	Implement Welcome page and Continue navigation	Execute TC-01	Aliya / Robert	Completed
Login	Implement simplified login workflow	Execute TC-02; record TC-03 and TC-04 as Omitted due to revised account-access scope.	Robert / Dev Team	Completed / Revised Scope
Profile Onboarding	Implement age, weight, gender, height, activity, and goal workflow	Execute TC-07, TC-08, and TC-13 to verify onboarding, measurement handling, and profile data storage.	Dev Team	Completed
Profile Management	Implement profile view/edit/save	Execute TC-06 and TC-14 to verify profile viewing and editing; record TC-15 as Omitted because profile deletion is outside the final scope.	Dev Team	Completed
BMI/BMR	Implement BMI/BMR calculations	Execute TC-09	Robert / Dev Team	Completed
Nutrition Goals	Implement calorie and macro goal settings	Execute TC-16–TC-21	Dev Team	Completed with limitations
Dashboard	Implement calorie/macro totals and progress display	Execute TC-05, TC-32, TC-33	Dev Team	Completed
Food Library	Implement add/edit/delete/sort/favorites/common foods	Execute TC-27–TC-29, TC-37–TC-45	Dev Team	Completed

Meal Logging	Implement log/edit/delete meal workflow	Execute TC-22–TC-26	Dev Team	Completed with limitation
Calendar / History	Implement date selection and daily history	Execute TC-30, TC-31, and TC-34–TC-36.	Dev Team	Completed
Multiple Profiles	Originally planned multi-profile workflow	TC-10–TC-12, TC-15 recorded as Omitted	Team	Omitted / Scope Change

7.Project Schedule

The following schedule reflects the original project timeline established during the planning phase. Final completion status, revised activities, and additional end-stage tasks are documented later in the Final Project Plan Update section.

Week	**Main Activities**
Week 1	Finalize project scope, confirm team roles, identify requirements, choose development tools
Week 2	Complete project planning, feasibility study, risk analysis, begin UI/UX wireframes and database design
Week 3	Finalize requirements, business rules, UML diagrams, database structure, and Figma designs
Week 4	Begin development of user profile, calorie goal system, and food database
Week 5	Develop meal logging, dashboard, and connect major application components
Week 6	Develop history/progress features and continue system integration
Week 7	Complete system integration, full prototype testing, bug fixing, validation, documentation, README, and testing evidence/screenshots
Week 8	Final review, presentation preparation and rehearsal, project packaging, and submission

The project schedule is organized so that planning and design are completed before major development begins. Development tasks may overlap when different team members are working on separate components. Testing will begin as individual features are completed and continue through the final weeks of the project.

Requirements must be finalized before database and UI design can be completed. User profile and food database development must be completed before full meal logging and dashboard integration. System integration must be completed before final system testing can occur.

8.Gantt Chart

Activity	W1	W2	W3	W4	W5	W6	W7	W8
Planning & Scope	■	■						
Requirements Analysis	■	■	■					
UI/UX Design in Figma		■	■					
Database Design		■	■					
UML / System Design		■	■					
User Profile & Calorie Goal				■	■			
Sample Food Database				■	■			
Meal Logging				■	■	■		
Dashboard					■	■		
History & Progress					■	■		
System Integration					■	■	■	
Feature Testing				■	■	■	■	
Full System Testing / Bug Fixing						■	■	■
Documentation						■	■	■
Final Presentation / Submission								■

Final Schedule Update

The project generally followed the planned eight-week schedule, with some implementation and testing tasks overlapping as development progressed. Final development work included correcting the Dashboard progress display, refining Food Library behavior, completing regression testing, cleaning the SQLite database for submission, updating installation instructions, and preparing the final project package. These activities were completed during the final development and submission phase.

9. Monitoring and Reporting Mechanisms

The team will use **Canvas group discussions** and **Discord** as the primary communication channels throughout the project. Canvas will be used for course-related group communication, while Discord will be used for faster communication, questions, updates, reminders, and coordination between team members.

The Project Manager will monitor the project schedule, assigned responsibilities, and overall progress. Team members will regularly report completed tasks, current work, and any problems that may affect the project schedule.

Google Docs will be used for file-based group assignments and shared project documents. This will allow multiple team members to work on the same document, review changes, and keep project information in one shared location.

GitHub will be used to store and manage the project code, track changes, and support collaboration between developers.

The team will also use the project schedule and Gantt chart to compare actual progress with planned progress. If delays or problems occur, the team will discuss them through Discord or team meetings and adjust responsibilities or the schedule when necessary.

Final Monitoring Update

During the final development phase, Discord remained the primary channel for day-to-day communication, while GitHub was used to manage branch-based development, code integration, and the final submission version. Progress was also monitored through shared testing results, requirement-based status tracking, and final review of completed documentation. These mechanisms allowed the team to coordinate remaining bug fixes, testing, database cleanup, and submission preparation.

10. Final Project Plan Update

The final stage of the Lifestyle Guru project focused on completing integration, validating the implemented requirements, correcting remaining defects, updating documentation, and preparing the software for submission. Several scope changes were made

during development to keep the project achievable within the course timeline, including removing multiple-profile support and the optional JSON-based recommendation feature. Final work also included correcting the Dashboard progress display, completing regression testing, cleaning the SQLite database so the application starts without previous test data, updating the README and installation instructions, and preparing the final GitHub branch and submission package.

Activity	Purpose	Responsible Member(s)	Status
System Integration	Integrate Flask, UI, SQLite, calculations	Dev Team	Completed
Regression Testing	Re-test after final fixes	Tri / Team	Completed
Full System Validation	Run final test set and collect screenshots	Tri / Team	Completed
Final Bug Fixing	Resolve progress display and other defects	Dev Team	Completed
Final Database Reset	Remove all test data before submission	Tri	Completed
README / Installation	Final setup/run instructions	Team	Completed
Final Packaging	Prepare final branch, ZIP, docs	Team	Completed

11. Appendix

The updated Appendix maps each major requirement to a specific implementation activity and a corresponding validation activity. It also identifies the responsible team member(s) and final completion status so that requirement development, testing, integration, and final delivery can be traced throughout the project.

Activity #	Requirement / Feature	Implementation Activity	Validation Activity	Responsible Member(s)	Final Status
1	Welcome Screen	Implement the Welcome screen and Continue navigation into the application.	Execute TC-01 to verify that the Welcome screen displays and navigation begin correctly.	Aliya / Development Team	Completed

Activity #	Requirement / Feature	Implementation Activity	Validation Activity	Responsible Member(s)	Final Status
2	Login / Account Access	Implement the simplified login workflow used by the prototype.	Execute TC-02 and TC-04 to verify the Login screen and application access.	Development Team	Completed / Revised Scope
3	Profile Onboarding	Implement the onboarding workflow for age, weight, gender, height, activity level, and goal information.	Execute TC-07, TC-08, and TC-13 to verify onboarding, measurement handling, profile data storage, and profile creation behavior.	Development Team	Completed with Scope Changes
4	Profile Management	Implement profile viewing, editing, and saving for the active profile.	Execute TC-06 and TC-14 to verify profile information and editing behavior. TC-15 is recorded as Omitted because profile deletion is outside the final project scope.	Development Team	Completed; Profile Deletion Omitted
5	BMI / BMR Calculations	Implement BMI and BMR calculations using saved profile information.	Execute TC-09 using valid profile values and confirm calculated results after saving.	Robert / Development Team	Completed with Limitation
6	Nutrition Goals	Implement calorie goals, protein/fat/carbohydrate percentages, macro allocation, editing, and reset behavior.	Execute TC-16–TC-21 to verify goal creation, 100% validation, calculations, editing, and reset behavior.	Development Team	Completed with Limitations
7	Dashboard / Progress	Implement calorie and macronutrient totals, goals, and visual progress indicators.	Execute TC-05 and TC-32–TC-33 to verify Dashboard components and progress behavior.	Development Team	Completed

Activity #	Requirement / Feature	Implementation Activity	Validation Activity	Responsible Member(s)	Final Status
8	Food Library	Implement custom food creation, persistence, sorting, editing, and deletion.	Execute TC-27–TC-29, TC-37–TC-38, and TC-43–TC-45 to verify Food Library operations.	Development Team	Completed
9	Favorites	Implement the ability to mark saved foods as favorites and filter the Food Library by favorite status.	Execute TC-39 and TC-40 to verify favorite status and Favorites list behavior.	Development Team	Completed
10	Most Common Foods	Implement food usage tracking and display frequently logged foods.	Execute TC-41 and TC-42 to verify Most Common Foods and access during meal logging.	Development Team	Completed
11	Meal Logging	Implement meal logging using saved Food Library items or newly created foods and assign meals to Breakfast, Lunch, Dinner, or Snacks.	Execute TC-22–TC-24 and TC-29 to verify meal entry options, categories, storage, and calorie calculations.	Development Team	Completed
12	Edit / Delete Logged Meals	Implement editing and deletion of saved meal records with recalculation of daily totals.	Execute TC-25 and TC-26 to verify meal modification and deletion behavior.	Development Team	Completed with Limitation
13	Calendar Navigation	Implement selectable date navigation for current, previous, and future dates.	Execute TC-30 and TC-34 to verify date navigation and day selection.	Development Team	Completed
14	Daily Nutrition History	Implement daily meal history, meal-category organization, and date-based nutrition totals.	Execute TC-31 and TC-35–TC-36 to verify stored meal history and daily totals.	Development Team	Completed
15	System Integration	Integrate the HTML/Jinja interface, Flask/Python application logic,	Perform integration and regression testing across the	Development Team	Completed

Activity #	Requirement / Feature	Implementation Activity	Validation Activity	Responsible Member(s)	Final Status
		calculations, and SQLite storage into one working prototype.	major workflows after components are combined.		
16	Full System Testing	Execute the final test-case set and record actual results for the completed prototype.	Re-run TC-01 through TC-45, document Pass / Pass with Limitation / Omitted results, and collect screenshot evidence.	Tri Phong / Team	Completed
17	Final Bug Fixing	Correct defects identified during final testing, including the Dashboard progress-display issue.	Re-test affected features after fixes to confirm that corrections do not break existing functionality.	Development Team	Completed
18	Final Documentation and Delivery	Finalize README instructions, User Documentation, Testing Evidence, clean database state, GitHub final branch, and submission package.	Perform a fresh-clone installation/run test and confirm the final package can be opened without previous test data.	Tri Phong / Team	Completed